

FINCORE BANK

Test Automation Training Programme

Document Type	Complete System Build Specification — Architecture, Data, API, UI, Pipeline
Primary Audience	Trainer / Architect / Vibe Coding Tool
Version	3.0 — Complete
Domain	Banking & Financial Services
Classification	Confidential — Do Not Share With Trainees

HOW TO USE THIS DOCUMENT

- This document is self-contained and complete. A vibe coding tool or developer receiving only this PDF has everything needed to build the full FinCore system.
- Section 1-3: Architecture, tech stack, prerequisites and Docker setup
- Section 4: Complete database schema with all constraints and rules
- Section 5: Data pipeline design, PySpark transformations, dataset sizes
- Section 6: Full application UI design — screens, charts, loaders, complexity
- Section 7: Complete REST API specification with sample JSON responses and error formats
- Section 8: Swagger/OpenAPI documentation generation
- Section 9: README and Comprehensive Guide specifications
- Section 10: GitHub repository structure
- Section 11: Use case overview for context

TABLE OF CONTENTS

- 1. Training Programme Overview**
 - 1.1 Objectives
 - 1.2 Team Structure
 - 1.3 Skill Coverage Map
- 2. System Architecture**
 - 2.1 High-Level Architecture
 - 2.2 Component Overview
 - 2.3 Docker Compose Specification
- 3. Technology Stack & Prerequisites**
 - 3.1 System Under Test Stack
 - 3.2 Test Automation Stack
 - 3.3 Prerequisites
- 4. Data Model & Schema**
 - 4.1 Entity Relationships
 - 4.2 Table Definitions — All Columns & Constraints
 - 4.3 Database Indexes
- 5. Data Pipeline & Dataset**
 - 5.1 Pipeline Overview
 - 5.2 PySpark Transformation Functions
 - 5.3 CSV Format Specification
 - 5.4 Dataset Sizes
 - 5.5 Good Data vs Bad Data — Violation Catalogue
- 6. Application UI Design**
 - 6.1 Design Principles & Testability Requirements
 - 6.2 Dashboard Screen
 - 6.3 Customers Screen
 - 6.4 Customer Detail Screen
 - 6.5 Transactions Screen
 - 6.6 Loans Screen
 - 6.7 Loan Detail Screen
 - 6.8 Shared UI Components
- 7. REST API Specification**
 - 7.1 API Design Principles
 - 7.2 Authentication
 - 7.3 Endpoint Specifications with Sample Responses
 - 7.4 Error Response Format
 - 7.5 Pagination Format
- 8. Swagger / OpenAPI Documentation**
 - 8.1 Setup
 - 8.2 Annotation Standard
 - 8.3 Accessing Swagger UI
- 9. README & Comprehensive Guide Specification**
 - 9.1 README.md — Required Sections
 - 9.2 comprehensive-guide.md — Required Sections
- 10. GitHub Repository Structure**
 - 10.1 Folder Layout

10.2 Key Files

11. Use Case Overview

11.1 UC Summary Table

11.2 Dependency Chain

1. TRAINING PROGRAMME OVERVIEW

1.1 Objectives

This training programme upskills Data Quality Engineers who already possess expert-level knowledge of Big Data ecosystems, PySpark, hyperscalers, and data warehouses. The programme introduces test automation — DQ validation, API automation, UI automation, pipeline testing, and CI/CD — using a realistic banking domain scenario called FinCore Bank.

1.2 Team Structure

Teams of 3. Ownership rotates per use case. Everyone touches every layer.

	UC1 — DQ	UC2 — API	UC3 — UI	UC4 — Pipeline	UC5 — Jenkins
Person A	GX Setup + Expectations	Transactions + Loans	Login + Dashboard	PySpark Unit Tests	Reports + Notify
Person B	Bad Data + Execution	Auth + Customers	Transactions + Customers	GX Validation	Jenkinsfile
Person C	Checkpoint + Data Docs	DB Cross-Validation	Loans + Screenshots	E2E Verification	Integration

1.3 Skill Coverage Map

Skill / Tool	UC1	UC2	UC3	UC4	UC5
Great Expectations (GX)	PRIMARY	Precondition	—	PRIMARY	Stage 1
PySpark	—	—	—	PRIMARY	—
pytest	Support	via pytest-bdd	via pytest-bdd	PRIMARY	—
pytest-bdd (Gherkin)	—	PRIMARY	PRIMARY	—	—
requests (API)	—	PRIMARY	—	Support	—
Selenium / Playwright	—	—	PRIMARY	Support	—
SQL / PostgreSQL	PRIMARY	PRIMARY	Support	PRIMARY	—
Jenkins	—	—	—	—	PRIMARY
pytest-html	Support	Support	Support	Support	PRIMARY
Git / GitHub	Support	Support	Support	Support	PRIMARY

2. SYSTEM ARCHITECTURE

2.1 High-Level Architecture

All components run locally via Docker Compose. The architecture is intentionally layered so each test layer has a distinct and meaningful role. 32 GB RAM handles all containers simultaneously.

```
CSV Files (good_data / bad_data)
  |
  v
PySpark Pipeline
- Reads CSVs - Transforms - Loads PostgreSQL
  |
  v
PostgreSQL 15 [customers | accounts | transactions | loans]
  |
+-----> Great Expectations (UC1, UC4)
  |
+-----> Node.js REST API :4000 (UC2)
  |      /api/v1/... + /api/docs (Swagger)
  |
+-----> React Web Portal :3000 (UC3)

Jenkins :8080 (UC5) orchestrates all test suites
```

2.2 Component Overview

Layer	Component	Technology	Port	Purpose
Source	CSV Files	Static files	—	Simulates nightly bank data dumps — good and bad variants
Pipeline	PySpark Job	PySpark 3.4+ / Python	—	Transforms and loads CSV data into PostgreSQL
Storage	PostgreSQL	PostgreSQL 15	5432	Central relational data store for all banking data
Validation	Great Expectations	GX 0.18+ / Python	—	Validates data quality post-pipeline load
API	FinCore REST API	Node.js 20 + Express	4000	Serves banking data as JSON. Swagger at /api/docs
UI	FinCore Web Portal	React 18	3000	Multi-view browser portal with grids, charts, loaders
CI/CD	Jenkins	Jenkins LTS	8080	Orchestrates all test suites end-to-end

2.3 Docker Compose Specification

The docker-compose.yml must define the following services exactly. Port mappings and environment variable names must match this spec — the test automation framework references them.

```
version: '3.8'
services:
```

```
postgres:
  image: postgres:15
  container_name: fincore-postgres
  ports: ['5432:5432']
  environment:
    POSTGRES_DB: fincore
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: fincore123
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
  healthcheck:
    test: ['CMD-SHELL', 'pg_isready -U admin -d fincore']
    interval: 10s
    retries: 5

fincore-app:
  build: ./app
  container_name: fincore-app
  ports: ['3000:3000', '4000:4000']
  depends_on:
    postgres: {condition: service_healthy}
  environment:
    DATABASE_URL: postgresql://admin:fincore123@postgres:5432/fincore
    JWT_SECRET: fincore_jwt_secret_2024
    JWT_EXPIRES_IN: 24h
    NODE_ENV: development
    API_PORT: 4000
    UI_PORT: 3000

jenkins:
  image: jenkins/jenkins:lts
  container_name: fincore-jenkins
  ports: ['8080:8080', '50000:50000']
  volumes:
    - jenkins_home:/var/jenkins_home
    - /var/run/docker.sock:/var/run/docker.sock
  depends_on: [fincore-app]

volumes:
  postgres_data:
  jenkins_home:
```

3. TECHNOLOGY STACK & PREREQUISITES

3.1 System Under Test Stack

Component	Technology	Version	Role
Web UI	React	18.x	Multi-view portal with data grids, SVG charts, skeleton loaders
State Management	React Context or Redux Toolkit	Latest	Manage async data loading states
Charts	Recharts or D3.js	Latest	SVG-based charts — bar, line, pie, donut, progress
UI Components	Tailwind CSS + custom	3.x	Responsive layout, badge components, skeleton loaders
Backend API	Node.js + Express	20.x LTS	REST API with Swagger documentation
API Docs	swagger-jsdoc + swagger-ui-express	Latest	Auto-generated Swagger UI at /api/docs
Database	PostgreSQL	15.x	Central relational store — 500K+ transaction records
Data Pipeline	PySpark	3.4+	Ingests and transforms large CSV datasets
Auth	JWT (jsonwebtoken)	9.x	Token-based authentication
ORM / Query	pg (node-postgres)	Latest	Direct PostgreSQL queries from Node.js

3.2 Test Automation Stack

Tool	Version	Purpose	Used In
Python	3.10+	Primary test language	All UCs
pytest	7.x+	Core test runner	UC1, UC4
pytest-bdd	6.x+	BDD Gherkin-based testing	UC2, UC3
Great Expectations	0.18+	Data Quality validation	UC1, UC4
requests	2.x	HTTP client for API calls	UC2
Selenium or Playwright	Latest	Browser automation	UC3
psycopg2-binary	2.9+	PostgreSQL connection	UC2, UC3, UC4
pytest-html	3.x+	HTML test reports	UC2, UC3, UC4

PySpark	3.4+	Unit test transformations	UC4
Jenkins	LTS	CI/CD orchestration	UC5

3.3 Prerequisites

Tool	Min Version	Verify	Notes
Python	3.10	python --version	Required for test automation
Docker Desktop	4.x	docker --version	Must support Compose v2
Docker Compose	2.x	docker compose version	Included in Docker Desktop
Git	2.x	git --version	GitHub account required
Java JDK	11 or 17	java -version	Required by PySpark
Node.js	20.x LTS	node --version	Required to build app outside Docker
IDE	Any	—	VS Code with Python + ESLint extensions recommended

4. DATA MODEL & SCHEMA

4.1 Entity Relationships

```
customers (1) ---< accounts (N)
accounts (1) ---< transactions (N)
customers (1) ---< loans (N)
```

4.2 Table Definitions

Table: customers

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-incremented unique identifier
name	VARCHAR(100)	NOT NULL	Full name — stored UPPERCASE after pipeline transformation
email	VARCHAR(150)	NOT NULL, UNIQUE	Customer email — used for communication and login
phone	VARCHAR(20)	NOT NULL	Contact number — numeric, 7-15 digits
address	TEXT	NULLABLE	Residential address — free text
date_of_birth	DATE	NOT NULL	Must be in the past — no future dates
status	VARCHAR(20)	NOT NULL	Allowed: active, inactive, blocked
created_date	TIMESTAMP	DEFAULT NOW()	Record creation timestamp
kyc_verified	BOOLEAN	DEFAULT FALSE	Know Your Customer verification flag

Table: accounts

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-incremented unique identifier
customer_id	INTEGER	FK -> customers.id NOT NULL	Owning customer
account_number	VARCHAR(20)	NOT NULL, UNIQUE	Bank account number — alphanumeric
account_type	VARCHAR(20)	NOT NULL	Allowed: savings, current, fixed_deposit
balance	DECIMAL(15,2)	NOT NULL	Current balance — savings accounts must be >= 0
currency	VARCHAR(3)	DEFAULT 'USD'	ISO 4217 currency code
status	VARCHAR(20)	NOT NULL	Allowed: active, dormant, closed

opened_date	DATE	NOT NULL	Date account was opened
-------------	------	----------	-------------------------

Table: transactions

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-incremented unique identifier
account_id	INTEGER	FK -> accounts.id NOT NULL	Account this transaction belongs to
transaction_type	VARCHAR(20)	NOT NULL	Allowed: credit, debit, transfer
amount	DECIMAL(15,2)	NOT NULL, CHECK > 0	Transaction value — must be positive
currency	VARCHAR(3)	DEFAULT 'USD'	ISO 4217 currency code
transaction_date	TIMESTAMP	NOT NULL	Date and time — must not be in the future
description	VARCHAR(255)	NULLABLE	Free-text transaction description
status	VARCHAR(20)	NOT NULL	Allowed: completed, pending, failed, reversed
reference_id	VARCHAR(50)	UNIQUE	External reference — must be unique

Table: loans

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-incremented unique identifier
customer_id	INTEGER	FK -> customers.id NOT NULL	Borrowing customer
loan_type	VARCHAR(30)	NOT NULL	Allowed: home, personal, auto, education
principal_amount	DECIMAL(15,2)	NOT NULL, CHECK > 0	Original loan amount disbursed
outstanding_amount	DECIMAL(15,2)	NOT NULL, CHECK >= 0	Remaining repayment amount
interest_rate	DECIMAL(5,2)	NOT NULL, CHECK 1-30	Annual interest rate percent
start_date	DATE	NOT NULL	Disbursement date
end_date	DATE	NOT NULL, CHECK start_date < end_date	Maturity date — must be after start_date
status	VARCHAR(20)	NOT NULL	Allowed: active, closed, defaulted, restructured

loan_duration_days	INTEGER	COMPUTED pipeline	by	end_date minus start_date in days
emi_amount	DECIMAL(15,2)	COMPUTED pipeline	by	Equated monthly instalment

4.3 Database Indexes

Index Name	Table	Column(s)	Reason
idx_customers_status	customers	status	Filter by status is frequent
idx_customers_email	customers	email	Unique lookups and search
idx_accounts_customer	accounts	customer_id	Join from customers
idx_transactions_account	transactions	account_id	Join from accounts — high volume table
idx_transactions_date	transactions	transaction_date	Date range filters on 500K rows
idx_transactions_status	transactions	status	Filter by status
idx_loans_customer	loans	customer_id	Join from customers
idx_loans_status	loans	status	Filter by status

5. DATA PIPELINE & DATASET

5.1 Pipeline Overview

The pipeline is a PySpark job (pipeline/ingest.py) that reads CSV files, applies transformations via functions in pipeline/transformations.py, and writes results to PostgreSQL. It is triggered via pipeline/run_pipeline.sh which accepts a data folder argument.

Usage:

```
bash pipeline/run_pipeline.sh good_data
bash pipeline/run_pipeline.sh bad_data
```

Flow:

1. Read all CSVs from data//
2. Apply transformations (see 5.2)
3. Truncate target tables
4. Write DataFrames to PostgreSQL via JDBC
5. Log row counts per table on completion

5.2 PySpark Transformation Functions

All functions live in pipeline/transformations.py. Trainees will unit test these in UC4 — write clear, testable, single-responsibility functions.

Function	Input	Output	Logic
standardise_name	DataFrame, column name	DataFrame	Convert name column to UPPERCASE using PySpark upper()
standardise_date	DataFrame, column name, input format	DataFrame	Parse date strings in DD/MM/YYYY to date type, reformat as YYYY-MM-DD
compute_loan_duration	DataFrame with start_date and end_date	DataFrame with loan_duration_days added	datediff(end_date, start_date) — returns integer days
compute_emi	DataFrame with principal, interest_rate, duration months	DataFrame with emi_amount added	Standard EMI formula: $P * r * (1+r)^n / ((1+r)^n - 1)$ where r = monthly rate
map_status_code	DataFrame, column name, code-to-label dict	DataFrame	Replace numeric status codes with string labels using PySpark replace()
fill_default_currency	DataFrame, column name	DataFrame	Fill NULL currency values with 'USD' using fillna()
filter_zero_amounts	DataFrame	DataFrame	Remove rows where amount = 0 using filter(col > 0)
trim_all_strings	DataFrame	DataFrame	Apply trim() to all StringType columns

5.3 CSV Format Specification

All CSVs use comma delimiter, UTF-8 encoding, with header row on line 1.

customers.csv:

```
id,name,email,phone,address,date_of_birth,status_code,created_date,kyc_verified
1,John Doe,john.doe@email.com,+1234567890,123 Main St,15/06/1985,1,2024-01-15,true

status_code mapping: 1=active 2=inactive 3=blocked
```

accounts.csv:

```
id,customer_id,account_number,account_type,balance,currency,status,opened_date
1,1,ACC001234,savings,15000.00,USD,active,2024-01-15
```

transactions.csv:

```
id,account_id,transaction_type,amount,currency,transaction_date,description,status,reference_id
1,1,credit,500.00,USD,2024-03-15 10:30:00,Salary deposit,completed,REF-20240315-001
```

loans.csv:

```
id,customer_id,loan_type,principal_amount,outstanding_amount,interest_rate,start_date,end_date,status
1,1,home,250000.00,235000.00,8.50,01/01/2023,01/01/2043,active

Note: loan_duration_days and emi_amount are NOT in CSV – pipeline computes them
```

5.4 Dataset Sizes

Use realistic large datasets. This is a Big Data training environment — trainees must experience working with meaningful data volumes.

Table	Record Count	Rationale
customers	10,000	Realistic mid-sized bank customer base
accounts	25,000	Average 2-3 accounts per customer
transactions	500,000	High volume — daily transactions over 2 years of history
loans	8,000	~80% of customers have at least one loan

Transactions are the largest table at 500,000 rows. Spread across 2 years of history (2023-2024). Mix of credit, debit, and transfer types. Statuses distributed: 70% completed, 15% pending, 10% failed, 5% reversed.

5.5 Good Data vs Bad Data — Violation Catalogue

The bad_data CSVs contain the following intentional violations. GX is expected to catch all of them. Do not tell trainees what violations exist — they must discover them through their expectations.

Table	Violation	Column	Count	GX Expectation
customers	NULL email	email	50 records	expect_column_values_to_not_be_null
customers	status = 'unknown'	status	30 records	expect_column_values_to_be_in_set

customers	Future date_of_birth	date_of_birth	20 records	expect_column_values_to_be_between
accounts	Negative balance on savings	balance	40 records	expect_column_values_to_be_between
accounts	account_type = 'premium'	account_type	25 records	expect_column_values_to_be_in_set
transactions	amount = negative values	amount	200 records	expect_column_values_to_be_between
transactions	Future transaction_date	transaction_date	150 records	expect_column_values_to_be_between
transactions	account_id not in accounts	account_id	100 records	expect_column_values_to_be_in_set
transactions	Duplicate reference_id	reference_id	80 pairs	expect_column_values_to_be_unique
loans	interest_rate = 95	interest_rate	35 records	expect_column_values_to_be_between
loans	end_date before start_date	end_date	25 records	Custom expectation
loans	NULL outstanding_amount	outstanding_amount	45 records	expect_column_values_to_not_be_null

6. APPLICATION UI DESIGN

6.1 Design Principles & Testability Requirements

The FinCore portal is a multi-view React application. Every interactive element must have a `data-testid` attribute for stable test automation. Avoid dynamic class names or index-based selectors. Charts must render as SVG elements with data attributes so test scripts can assert values. All async data loads must show a skeleton loader before data appears.

TESTABILITY REQUIREMENTS — MANDATORY FOR ALL SCREENS

- Every button: `data-testid='--btn'` e.g. `data-testid='login-submit-btn'`
- Every input: `data-testid='--input'` e.g. `data-testid='login-username-input'`
- Every table: `data-testid='--table'` — rows as `data-testid='--row'`
- Every filter/dropdown: `data-testid='--filter'`
- Every chart: `data-testid='--chart'` with `data-value` attributes on SVG elements
- Every skeleton loader: `data-testid='--skeleton-loader'`
- Every summary card: `data-testid='dashboard--card'` with `data-value` attribute
- Every error message: `data-testid='--error-message'`

6.2 Dashboard Screen — Route: /dashboard

Element		Type	Data Source	Testable Behaviour
Total card	Customers	Summary card	SELECT COUNT(*) FROM customers	<code>data-testid='dashboard-total-customers-card'</code> <code>data-value=""</code>
Active card	Accounts	Summary card	SELECT COUNT(*) FROM accounts WHERE status='active'	<code>data-testid='dashboard-active-accounts-card'</code> <code>data-value=""</code>
Transactions Today card		Summary card	SELECT COUNT(*) FROM transactions WHERE DATE(transaction_date)=TODAY	<code>data-testid='dashboard-transactions-today-card'</code>
Active Loans card		Summary card	SELECT COUNT(*) FROM loans WHERE status='active'	<code>data-testid='dashboard-active-loans-card'</code> <code>data-value=""</code>
Transactions by Day chart		SVG Bar Chart	Last 30 days transaction counts grouped by date	<code>data-testid='dashboard-bar-chart'</code> — each bar has <code>data-date</code> and <code>data-value</code>
Loan Status chart		SVG Pie Chart	COUNT(*) FROM loans GROUP BY status	<code>data-testid='dashboard-pie-chart'</code> — each slice has <code>data-status</code> and <code>data-value</code>
Skeleton loaders		Loader	Shown during initial API fetch	<code>data-testid='dashboard-skeleton-loader'</code> visible then disappears

6.3 Customers Screen — Route: /customers

Element	Type	Behaviour	Testable
Customer data grid	Paginated table	Columns: ID, Name, Email, Phone, Status, KYC, Created Date. 25 rows per page	data-testid='customers-table', rows have data-testid='customers-row-'
Search box	Text input	Filters by name or email — debounced 300ms	data-testid='customers-search-input'
Status filter	Dropdown	Options: All, Active, Inactive, Blocked	data-testid='customers-status-filter'
Column sort	Click header	Sort ASC/DESC on Name, Status, Created Date	Column headers have data-testid='customers-sort-'
Pagination	Page controls	Previous, Next, page numbers — shows total record count	data-testid='customers-pagination'
Row click	Navigation	Clicking a row navigates to /customers/:id	Row click triggers route change
Table skeleton	Loader	Shown while data loads	data-testid='customers-skeleton-loader'

6.4 Customer Detail Screen — Route: /customers/:id

Element	Type	Behaviour	Testable
Customer info card	Detail card	Shows all customer fields — name, email, phone, DOB, status, KYC badge	data-testid='customer-detail-card'
Tab: Accounts	Tab panel	Sub-grid of accounts linked to this customer — lazy loaded on tab click	data-testid='customer-accounts-tab', table data-testid='customer-accounts-table'
Tab: Transactions	Tab panel	Recent 50 transactions across all accounts — lazy loaded	data-testid='customer-transactions-tab'
Transaction history chart	SVG Line Chart	Transaction amounts over time for this customer	data-testid='customer-linechart' — points have data-date data-value
Tab skeleton loader	Loader	Each tab shows skeleton until its data loads	data-testid='customer-tab-skeleton'
Back button	Navigation	Returns to /customers	data-testid='customer-back-btn'

6.5 Transactions Screen — Route: /transactions

Element	Type	Behaviour	Testable
---------	------	-----------	----------

Transaction grid	Paginated table	Columns: ID, Account, Type, Amount, Currency, Date, Status, Reference. 50 rows per page	data-testid='transactions-table'
Type filter	Dropdown	All, Credit, Debit, Transfer	data-testid='transactions-type-filter'
Status filter	Dropdown	All, Completed, Pending, Failed, Reversed	data-testid='transactions-status-filter'
Date range picker	Date inputs	From date and To date — filters transaction_date	data-testid='transactions-from-date' and 'transactions-to-date'
Amount range	Number inputs	Min amount and Max amount	data-testid='transactions-min-amount' and 'transactions-max-amount'
Apply filters btn	Button	Applies all active filters simultaneously	data-testid='transactions-apply-filters-btn'
Clear filters btn	Button	Resets all filters to default	data-testid='transactions-clear-filters-btn'
Volume by type chart	SVG Bar Chart	Count of transactions grouped by type	data-testid='transactions-bar-chart'
Table skeleton	Loader	Shown during load and filter application	data-testid='transactions-skeleton-loader'

6.6 Loans Screen — Route: /loans

Element	Type	Behaviour	Testable
Loans grid	Paginated table	Columns: ID, Customer, Type, Principal, Outstanding, Rate, Status, Start, End. 25 rows per page	data-testid='loans-table'
Status filter	Dropdown	All, Active, Closed, Defaulted, Restructured	data-testid='loans-status-filter'
Type filter	Dropdown	All, Home, Personal, Auto, Education	data-testid='loans-type-filter'
Status badges	Coloured badge	Active=green, Closed=grey, Defaulted=red, Restructured=orange	data-testid='loan-status-badge-' with data-status attribute
Status distribution chart	SVG Donut Chart	Loan count by status	data-testid='loans-donut-chart' — each segment has data-status data-value

Row expand	Expandable row	Click row to expand inline showing EMI and duration without navigating away	data-testid='loans-expand-row'
View detail link	Navigation	Link in expanded row to /loans/:id full detail page	data-testid='loans-view-detail'

6.7 Loan Detail Screen — Route: /loans/:id

Element	Type	Behaviour	Testable
Loan detail card	Detail card	All loan fields including loan_type, principal, outstanding, rate, start, end, status	data-testid='loan-detail-card'
Computed fields section	Info section	loan_duration_days and emi_amount displayed prominently	data-testid='loan-duration-value' and 'loan-emi-value' with data-value attribute
Repayment progress bar	SVG Progress	Visual bar showing (principal - outstanding) / principal as percentage	data-testid='loan-repayment-progress' with data-percentage attribute
Full page loader	Loader	Shown while loan data fetches	data-testid='loan-detail-skeleton'
Customer link	Navigation	Links back to /customers/:customer_id	data-testid='loan-customer-link'

6.8 Shared UI Components

Component	Behaviour	Used On
Navigation sidebar	Links to all screens. Highlights active route. Collapsible on mobile	All authenticated screens
Top header bar	Shows current user, logout button, app name	All authenticated screens
Login page	Username + password inputs, submit button, error message area. Route: /login	Unauthenticated state
404 page	Shown for unknown routes. Link back to dashboard	Invalid routes
Toast notifications	Success/error toasts on API failures	All screens
Skeleton loader component	Grey pulsing placeholder matching shape of content	All data-loading states
Status badge component	Coloured pill badge for status fields	Customers, Transactions, Loans screens

7. REST API SPECIFICATION

7.1 API Design Principles

- Base URL: `http://localhost:4000/api/v1`
- All responses are JSON with `Content-Type: application/json`
- All endpoints except `/auth/login` and `/health` require `Authorization: Bearer`
- Successful responses use 200 OK for reads
- All list endpoints support pagination via `?page=1&limit=25`
- All list endpoints return `{ data: [...], total: N, page: N, limit: N }`
- Error responses always include `{ error: "", code: "" }`

7.2 Authentication

Username	Password	Role	Access Level
admin	Admin@123	admin	Full read access to all endpoints
viewer	Viewer@123	read_only	Read-only — same as admin for GET requests
testuser	Test@123	standard	Standard — use for negative auth test scenarios

POST `/api/v1/auth/login` — Request and Response:

```
REQUEST:
POST /api/v1/auth/login
Content-Type: application/json
{
  "username": "admin",
  "password": "Admin@123"
}

RESPONSE 200:
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "username": "admin",
    "role": "admin"
  },
  "expires_in": "24h"
}

RESPONSE 401 (invalid credentials):
{
  "error": "Invalid username or password",
  "code": "AUTH_INVALID_CREDENTIALS"
}
```

7.3 Endpoint Specifications with Sample Responses

GET `/api/v1/health`

```
RESPONSE 200:
{
```

```
"status": "ok",
"db": "connected",
"timestamp": "2024-03-15T10:30:00.000Z",
"version": "1.0.0"
}
```

GET /api/v1/customers

QUERY PARAMS: ?status=active&search;=john&page;=1&limit;=25

RESPONSE 200:

```
{
  "data": [
    {
      "id": 1,
      "name": "JOHN DOE",
      "email": "john.doe@email.com",
      "phone": "+1234567890",
      "address": "123 Main St",
      "date_of_birth": "1985-06-15",
      "status": "active",
      "kyc_verified": true,
      "created_date": "2024-01-15T00:00:00.000Z"
    }
  ],
  "total": 10000,
  "page": 1,
  "limit": 25
}
```

GET /api/v1/customers/:id

RESPONSE 200:

```
{
  "customer": { "id":1, "name":"JOHN DOE", "email":"john.doe@email.com",
                "status":"active", "kyc_verified":true },
  "accounts": [
    { "id":1, "account_number":"ACC001234", "account_type":"savings",
      "balance":15000.00, "status":"active" }
  ],
  "recent_transactions": [
    { "id":1, "transaction_type":"credit", "amount":500.00,
      "transaction_date":"2024-03-15T10:30:00.000Z", "status":"completed" }
  ]
}
```

RESPONSE 404:

```
{ "error": "Customer not found", "code": "CUSTOMER_NOT_FOUND" }
```

GET /api/v1/transactions

QUERY PARAMS: ?account_id=1&type;=credit&status;=completed
&from;_date=2024-01-01&to;_date=2024-06-30
&min;_amount=100&max;_amount=5000&page;=1&limit;=50

RESPONSE 200:

```
{
  "data": [
    {
      "id": 1001,
      "account_id": 1,
      "account_number": "ACC001234",

```

```

    "transaction_type": "credit",
    "amount": 500.00,
    "currency": "USD",
    "transaction_date": "2024-03-15T10:30:00.000Z",
    "description": "Salary deposit",
    "status": "completed",
    "reference_id": "REF-20240315-001"
  }
],
"total": 500000,
"page": 1,
"limit": 50
}

```

GET /api/v1/loans/:id

RESPONSE 200:

```

{
  "id": 1,
  "customer_id": 1,
  "customer_name": "JOHN DOE",
  "loan_type": "home",
  "principal_amount": 250000.00,
  "outstanding_amount": 235000.00,
  "interest_rate": 8.50,
  "start_date": "2023-01-01",
  "end_date": "2043-01-01",
  "status": "active",
  "loan_duration_days": 7305,
  "emi_amount": 2152.39,
  "repayment_percentage": 6.00
}

```

GET /api/v1/dashboard/summary

RESPONSE 200:

```

{
  "total_customers": 10000,
  "active_accounts": 18500,
  "transactions_today": 1243,
  "active_loans": 5600,
  "queries_used": {
    "total_customers": "SELECT COUNT(*) FROM customers",
    "active_accounts": "SELECT COUNT(*) FROM accounts WHERE status='active'",
    "transactions_today": "SELECT COUNT(*) FROM transactions WHERE DATE(transaction_date)=CURRENT_DATE",
    "active_loans": "SELECT COUNT(*) FROM loans WHERE status='active'"
  }
}

```

7.4 Error Response Format

All error responses must follow this exact structure:

400 Bad Request:

```
{ "error": "Validation failed: is required", "code": "VALIDATION_ERROR" }
```

401 Unauthorized:

```
{ "error": "Authentication required", "code": "AUTH_REQUIRED" }
```

```
{ "error": "Token expired", "code": "TOKEN_EXPIRED" }
```

```
{ "error": "Invalid username or password", "code": "AUTH_INVALID_CREDENTIALS" }
```

```
404 Not Found:
{ "error": "Customer not found", "code": "CUSTOMER_NOT_FOUND" }
{ "error": "Loan not found", "code": "LOAN_NOT_FOUND" }

500 Internal Server Error:
{ "error": "Internal server error", "code": "INTERNAL_ERROR" }
```

7.5 Pagination Format

All list endpoints must implement and return pagination consistently:

Query params: `?page=1&limit=25` (defaults: `page=1, limit=25`)

Response envelope for all list endpoints:

```
{
  "data": [...],           // Array of records for current page
  "total": 10000,          // Total records matching the query (ignoring pagination)
  "page": 1,               // Current page number
  "limit": 25,             // Records per page
  "total_pages": 400       // Computed: ceil(total / limit)
}
```

8. SWAGGER / OPENAPI DOCUMENTATION

8.1 Setup

The Node.js API must auto-generate Swagger documentation using `swagger-jsdoc` and serve it via `swagger-ui-express`. The Swagger UI must be accessible at `/api/docs`. This allows trainees to explore and manually test all API endpoints interactively.

```
npm install swagger-jsdoc swagger-ui-express

// In app.js or server.js:
const swaggerJsdoc = require('swagger-jsdoc');
const swaggerUi = require('swagger-ui-express');

const swaggerOptions = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'FinCore Bank API',
      version: '1.0.0',
      description: 'FinCore Bank REST API for test automation training'
    },
    servers: [{ url: 'http://localhost:4000/api/v1' }],
    components: {
      securitySchemes: {
        bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' }
      }
    }
  },
  apis: ['./src/routes/*.js']
};

const swaggerSpec = swaggerJsdoc(swaggerOptions);
app.use('/api/docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
app.get('/api/docs.json', (req, res) => res.json(swaggerSpec));
```

8.2 Annotation Standard

Every route file must include JSDoc annotations in the following format. This ensures the auto-generated Swagger UI is complete and usable by trainees.

```
/**
 * @swagger
 * /customers:
 *   get:
 *     summary: Get all customers with optional filters
 *     security:
 *       - bearerAuth: []
 *     parameters:
 *       - in: query
 *         name: status
 *         schema: { type: string, enum: [active, inactive, blocked] }
 *       - in: query
 *         name: search
 *         schema: { type: string }
 *       - in: query
 *         name: page
 *         schema: { type: integer, default: 1 }
```

```

*      - in: query
*        name: limit
*        schema: { type: integer, default: 25 }
*    responses:
*      200:
*        description: Paginated customer list
*      401:
*        description: Unauthorized
*/

```

8.3 Accessing Swagger UI

URL	Purpose
http://localhost:4000/api/docs	Interactive Swagger UI — explore and test all endpoints
http://localhost:4000/api/docs.json	Raw OpenAPI JSON spec — import into Postman or other tools

9. README & COMPREHENSIVE GUIDE SPECIFICATION

Two documentation files must be created in the repository root. README.md is the quick-start guide — get running in 10 minutes. comprehensive-guide.md is the deep-dive — understand everything.

9.1 README.md — Required Sections

README.md must be concise and action-oriented. No theory — only what is needed to get the system running. Everything that could change (ports, credentials, commands) lives here, not in this PDF.

Section	Content Required
Prerequisites	Exact software list with minimum versions and install links
Quick Start	Step-by-step: clone, docker compose up, run pipeline, verify — numbered, one command per step
Port Reference	Table: Service Port URL Purpose for all 4 services (Postgres, App UI, App API, Jenkins)
Credentials	Table: Service Username Password for all test accounts
Running the Pipeline	Exact commands for good_data and bad_data runs with expected output
Verifying the System	How to confirm each service is healthy — curl commands or browser URLs
Running Tests	One-line commands to run each test suite independently
Stopping the System	docker compose down command and data cleanup
Troubleshooting	5 most common issues with solutions — port conflicts, memory, pipeline failures

9.2 comprehensive-guide.md — Required Sections

The comprehensive guide is for anyone who wants to understand the system deeply — not just run it. Trainees can read this for context but it is not required for the exercises.

Section	Content Required
Architecture Overview	Diagram and explanation of all components and how they connect
Technology Stack	Why each technology was chosen — rationale not just list
Data Model	Full schema with ER diagram description and relationship explanations
Pipeline Deep Dive	How PySpark reads, transforms, and loads — step by step with transformation logic
API Design	REST design decisions, authentication flow, error handling approach
UI Design	Screen-by-screen description with component choices and UX decisions
Environment Variables	Complete list of all env vars with descriptions and defaults
Dataset Generation	How the CSVs were generated — tools used, data distribution decisions

Adding New Data	How to regenerate or extend the datasets
FAQ	At least 10 common questions with answers

10. GITHUB REPOSITORY STRUCTURE

10.1 Folder Layout

```
fincore-bank/
|
+-- README.md                # Quick start – prerequisites, commands, ports
+-- comprehensive-guide.md   # Deep dive – architecture, data model, FAQ
+-- docker-compose.yml       # All services: Postgres, App, Jenkins
|
+-- pipeline/
|   +-- ingest.py            # Main PySpark job
|   +-- transformations.py   # All transformation functions (trainees unit test these)
|   +-- requirements.txt      # PySpark, psycopg2, etc.
|   +-- run_pipeline.sh      # bash run_pipeline.sh good_data | bad_data
|
+-- data/
|   +-- good_data/
|   |   +-- customers.csv    # 10,000 records – clean
|   |   +-- accounts.csv    # 25,000 records – clean
|   |   +-- transactions.csv # 500,000 records – clean
|   |   +-- loans.csv       # 8,000 records – clean
|   +-- bad_data/
|   |   +-- customers.csv    # 10,000 records with injected violations
|   |   +-- accounts.csv    # 25,000 records with injected violations
|   |   +-- transactions.csv # 500,000 records with injected violations
|   |   +-- loans.csv       # 8,000 records with injected violations
|
+-- app/
|   +-- Dockerfile
|   +-- package.json
|   +-- src/
|   |   +-- server.js        # Express API entry point + Swagger setup
|   |   +-- routes/          # One file per resource + JSDoc Swagger annotations
|   |   |   +-- auth.js
|   |   |   +-- customers.js
|   |   |   +-- accounts.js
|   |   |   +-- transactions.js
|   |   |   +-- loans.js
|   |   |   +-- dashboard.js
|   |   +-- middleware/
|   |   |   +-- auth.js      # JWT verification middleware
|   |   +-- client/          # React app (build output served by Express)
|
+-- db/
|   +-- init.sql             # Schema + indexes (auto-runs on Postgres start)
|
+-- tests/                   # TRAINEES WORK HERE
|   +-- dq/                  # UC1 - Great Expectations
|   +-- api/                 # UC2 - pytest-bdd
|   +-- ui/                  # UC3 - pytest-bdd
|   +-- pipeline/            # UC4 - PySpark + E2E
|   +-- Jenkinsfile          # UC5
|   +-- conftest.py
|   +-- requirements.txt
|   +-- reports/
```

11. USE CASE OVERVIEW

11.1 UC Summary Table

UC	Title	Primary Tools	Deliverables
UC1	Data Quality Validation	Great Expectations, PostgreSQL, Python	GX Expectation Suites, Checkpoint, Data Docs HTML reports (good + bad run), Python trigger script
UC2	API Automation	pytest-bdd, requests, psycopg2, PostgreSQL	Gherkin feature files, step definitions, DB cross-validation, pytest-html report
UC3	UI Automation	pytest-bdd, Selenium or Playwright, psycopg2	Gherkin feature files, step definitions, failure screenshots, pytest-html report
UC4	Pipeline Testing	PySpark, pytest, GX, requests, Selenium/Playwright	PySpark unit tests, extended GX suites, E2E test script, good + bad data reports
UC5	Jenkins CI/CD	Jenkins, Jenkinsfile, all UC1-UC4 scripts	Jenkinsfile, configured Jenkins job, unified reports, failure notification demo

11.2 Dependency Chain

```
UC1: GX validates PostgreSQL data quality
--> UC2: API tests run (UC1 pass is precondition)
    --> UC3: UI tests run (API tests pass is precondition)
        --> UC4: Full pipeline tested using UC1+UC2+UC3 techniques
            --> UC5: Jenkins runs UC1->UC2->UC3->UC4 in one automated pipeline
```

One bank. One dataset. One pipeline. Five layers of testing.